

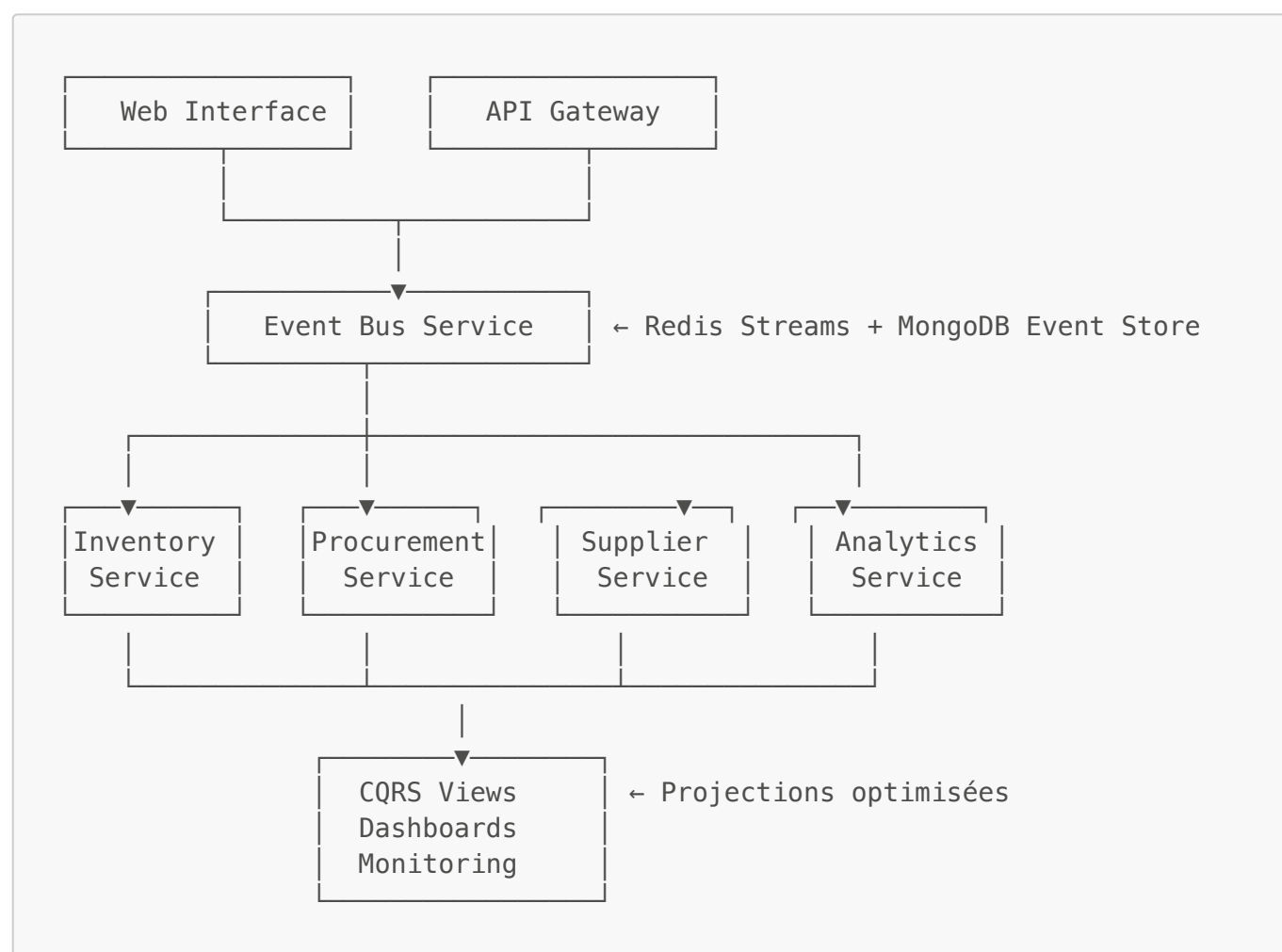
Lab 7 - Architecture Événementielle : Event Sourcing, CQRS, Pub/Sub et Saga

🎯 Objectif du Laboratoire

Ce laboratoire implémente une **architecture microservices événementielle complète** utilisant les patterns avancés :

- **Event Sourcing** : Persistance des événements comme source de vérité
- **CQRS** : Séparation Command/Query avec projections optimisées
- **Pub/Sub** : Communication asynchrone découplée
- **Saga Chorégraphiée** : Gestion des transactions distribuées

🏗️ Architecture Globale



🚀 Démarrage Ultra-Rapide

Option 1: Script Automatisé (Recommandé)

```
# Cloner et démarrer tout automatiquement
git clone https://github.com/zakzaki244/Laboratoires-LOG430/tree/lab7
```

```
cd Laboratoires-L0G430-1

# Démarrage complet avec un seul script
./run_lab7.sh start

# Attendre 30 secondes puis tester
./run_lab7.sh test
```

Option 2: Démarrage Manuel

```
# 1. Infrastructure événementielle
docker-compose -f event-infrastructure.yml up -d

# 2. Tous les services Lab 7
docker-compose -f docker-compose-lab7.yml up -d

# 3. Initialiser les données
python init_lab7_data.py

# 4. Vérifier le déploiement
python test_event_driven_architecture.py
```

 Services et Ports

Service	Port	Rôle	Technologie
Event Bus Service	5000	Hub central événements	Flask + Redis Streams + MongoDB
Inventory Service	5001	Gestion stock événementielle	Flask + Event Sourcing
Procurement Service	5002	Commandes fournisseurs	Flask + Saga Pattern
Supplier Service	5003	Simulation fournisseurs	Flask + Async Processing
Analytics Service	5007	Projections CQRS	Flask + MongoDB Views
API Gateway	8080	Agrégation APIs	Flask + Service Discovery
Web Interface	5008	Dashboard temps réel	Flask + WebSocket
Prometheus	9090	Métriques	Prometheus
Grafana	3000	Visualisation	Grafana (admin/admin)

 Patterns Événementiels Implémentés

1. Event Sourcing

```
# Tous les changements = événements persistés
events = [
```

```

    {"type": "ProductSold", "data": {"product_id": "P123", "qty": 5}},
    {"type": "StockUpdated", "data": {"product_id": "P123", "new_qty":
15}},
    {"type": "LowStockDetected", "data": {"product_id": "P123",
"threshold": 10}}
]
# État reconstruit par replay des événements

```

2. CQRS (Command Query Responsibility Segregation)

```

# Commands → Event Bus (écritures)
POST /events/publish {"event_type": "ProductSold", "data": "..."}

# Queries → Analytics Service (lectures optimisées)
GET /projections/stock      # Vue stock temps réel
GET /projections/sales      # Analytics ventes
GET /projections/dashboard  # Tableau de bord complet

```

3. Saga Chorégraphiée - Flux de Réapprovisionnement

```

1. LowStockDetected      → Procurement Service
2. RestockApproved       → Supplier Service
3. SupplierDelivered     → Inventory Service
4. StockUpdated          → Analytics Service

En cas d'échec: Événements de compensation automatiques

```

4. Pub/Sub avec Redis Streams

```

# Publication
redis.xadd('events', {'event_type': 'ProductSold', 'data':
json.dumps(data)})

# Consommation avec groupes
redis.xreadgroup('analytics-group', 'consumer-1', {'events': '>'})

```

Tests Automatisés

Tests d'Architecture Événementielle

```
python test_event_driven_architecture.py
```

Vérifie :

-  Publication et consommation d'événements
-  Persistance Event Store MongoDB
-  Projections CQRS temps réel
-  Saga de réapprovisionnement complète
-  Replay d'événements pour debug

Tests de Scénarios d'Échec

```
python test_failure_scenarios.py
```

Teste la résilience :

-  Timeout fournisseur →  Compensation automatique
-  Stock insuffisant →  Recherche fournisseur alternatif
-  Budget dépassé →  Rejet avec audit trail
-  Service indisponible →  Circuit breaker

Tests d'Intégration Complète

```
python test_integration_complete.py
```

Valide bout-en-bout :

-  API Gateway routing
-  Web Interface dashboard
-  Projections CQRS via API
-  Performance et latence



Monitoring et Observabilité

Dashboard Grafana (<http://localhost:3000>)

- **Flux d'événements** en temps réel
- **États des sagas** et compensations
- **Performance des services** (latence, débit)
- **Santé système** et alertes

Métriques Prometheus (<http://localhost:9090>)

- **events_processed_total** - Événements traités
- **saga_transactions_total** - Transactions distribuées
- **stock_level** - Niveaux de stock
- **supplier_delivery_time_avg** - Performance fournisseurs

Analytics Dashboard (<http://localhost:5007/projections/dashboard>)

```
{
  "stock_summary": {
    "total_products": 25,
    "low_stock_count": 3,
    "active_alerts": 1
  },
  "sales_summary": {
    "total_sales": 1247,
    "revenue": 48293.75,
    "top_products": {"LAPTOP_001": 45, "MOUSE_002": 128}
  },
  "system_health": {
    "events_processed": 15847,
    "services_status": "all_healthy"
  }
}
```

Commandes Utiles

Script d'Automatisation

```
./run_lab7.sh start          # Démarrage complet
./run_lab7.sh test           # Tous les tests
./run_lab7.sh health         # Vérifier santé services
./run_lab7.sh monitor        # Monitoring temps réel
./run_lab7.sh logs [service] # Voir les logs
./run_lab7.sh cleanup        # Nettoyage
./run_lab7.sh help           # Aide complète
```

Tests Manuels via API

```
# Publier un événement de vente
curl -X POST http://localhost:5000/events/publish \
  -H "Content-Type: application/json" \
  -d '{"event_type": "ProductSold", "data": "{\"product_id\": \"LAPTOP_001\", \"quantity\": 3}\"}'

# Consulter les projections CQRS
curl http://localhost:5007/projections/dashboard | jq

# Voir le flux d'événements
curl http://localhost:5000/events/stream | jq
```

Exemple de Saga Complète

Scénario : Stock Bas → Réapprovisionnement Automatique

1. Déclencheur : Vente fait passer le stock sous le seuil

```
curl -X POST http://localhost:5001/inventory/sell \
-d '{"product_id": "LAPTOP_001", "quantity": 8}'
```

2. Événements générés automatiquement :

StockUpdated → LowStockDetected → RestockApproved → SupplierDelivered → StockUpdated

3. Suivi en temps réel :

```
# Voir la saga en cours
curl http://localhost:5007/projections/movements

# Monitoring continu
watch -n 2 "curl -s http://localhost:5007/projections/dashboard | jq
.system_summary"
```

Troubleshooting

Problèmes Courants

1. Services ne démarrent pas

```
# Vérifier Docker
docker --version && docker-compose --version

# Vérifier les ports occupés
lsof -i :5000-5010

# Logs détaillés
./run_lab7.sh logs event-bus-service
```

2. Événements non traités

```
# État Redis Streams
docker exec -it redis-events redis-cli XINFO STREAM events

# Groupes de consommateurs
docker exec -it redis-events redis-cli XINFO GROUPS events
```

3. Projections désynchronisées

```
# Reconstruire les projections
curl -X POST http://localhost:5007/projections/rebuild
```

Procédures de Test - Succès et Échec de la Saga

Test de Succès Complet

```
# 1. Démarrer le système
./run_lab7.sh start

# 2. Tester le scénario de succès automatiquement
./demo_lab7.sh

# 3. Vérifier manuellement la saga de réapprovisionnement
curl -X POST "http://localhost:5000/events/publish" \
  -H "Content-Type: application/json" \
  -d '{
    "event_type": "ProductSold",
    "aggregate_id": "product_LAPTOP_001",
    "aggregate_type": "Product",
    "data": {
      "product_id": "LAPTOP_GAMING_001",
      "quantity": 25,
      "customer_id": "customer_test_001",
      "price": 1299.99
    }
  }'
```

4. Observer la séquence d'événements (stock bas → réapprovisionnement → succès)

```
curl http://localhost:5000/events | jq '.events[] | select(.correlation_id != null)'
```

Test d'Échec et Compensation

```
# 1. Simuler un échec de fournisseur
curl -X POST "http://localhost:5003/simulate/failure" \
  -H "Content-Type: application/json" \
  -d '{"failure_type": "supplier_unavailable", "duration": 30}'

# 2. Déclencher une commande qui va échouer
curl -X POST "http://localhost:5000/events/publish" \
  -H "Content-Type: application/json" \
  -d '{
    "event_type": "LowStockDetected",
    "aggregate_id": "inventory_LAPTOP_001",
```

```
    "data": {
      "product_id": "LAPTOP_GAMING_001",
      "current_quantity": 5,
      "threshold": 10,
      "requested_quantity": 50
    }
  }'
```

3. Observer les événements de compensation

```
tail -f logs/procurement-service.log | grep "compensation"
```

4. Vérifier les événements compensatoires dans l'Event Store

```
curl "http://localhost:5000/events?event_type=RestockCancelled"
```

Test de Résilience et Replay

1. Simuler une panne de service

```
docker stop procurement-service
```

2. Publier des événements pendant la panne

```
for i in {1..5}; do
```

```
  curl -X POST "http://localhost:5000/events/publish" \
```

```
    -H "Content-Type: application/json" \
```

```
    -d '{"event_type": "TestEvent", "data": {"test_id": $i}}'
```

```
done
```

3. Redémarrer le service

```
docker start procurement-service
```

4. Vérifier le replay automatique

```
curl "http://localhost:5002/health" && echo "Service recovered"
```

5. Replay manuel depuis une position

```
curl -X POST "http://localhost:5000/events/replay" \
```

```
  -H "Content-Type: application/json" \
```

```
  -d '{"from_timestamp": "2025-08-07T00:00:00Z", "service":
    "procurement-service"}'
```

Tests Automatisés Complets

Tests d'architecture événementielle

```
python test_event_driven_architecture.py
```

Tests de scénarios d'échec

```
python test_failure_scenarios.py
```

Tests d'intégration complète

```
python test_integration_complete.py
```



```
# Rapport de synthèse des tests  
./run_lab7.sh test-report
```

Vérification des Métriques

```
# Métriques Prometheus  
curl "http://localhost:9090/api/v1/query?query=events_published_total"  
  
# Dashboard Grafana  
open http://localhost:3000 # login: admin/admin  
  
# Logs en temps réel  
./run_lab7.sh monitor
```